UNTRM · INCID · FIZBAC · 2026-II

De los datos al modelo

OPERACIONAL → RAW → MASTER → FS → ML, con un cuaderno por capa y un modelo que se puede explicar.

CASO: ONLINE RETAIL II | PREGUNTA: ¿VOLVERÁ EN 90 DÍAS? | MLFLOW · TRACKING

Lo que construimos hoy

1 067 371 filas → 38 404 ejemplos

de un sistema de ventas con suciedad real a una tabla lista para entrenar, sin usar información del futuro.

AGENDA

Cuatro horas, cuatro bloques



0:00 · Revisión (30 min)

Cada quien comprueba `env-ia-mlops` con la prueba de humo. Quien falla, trabaja con Codespaces o en pareja.

0:30 · Teoría (60 min)

Capas, linaje, calidad, regla de oro de FS, partición temporal, entrenamiento y MLflow.

1:30 · Práctica (135 min)

Los cinco cuadernos (00 a 04), con pausa de 10 min. Termina con la traducción a `.py` y las pruebas.

3:45 · Entrega (15 min)

Se explica la tarea y la rúbrica, se resuelven dudas y se avanza a S2.

REVISIÓN · 30 MIN

¿Funciona tu entorno?

```
conda activate env-ia-mlops
cd 04-Proyecto-Base
python ../03-Entorno/conda/prueba_de_humo.py
python -m pytest -q
```

> Esperado: la prueba de humo termina sin errores y **pytest** informa `123 passed`.

Si algo falla

- **conda pide aceptar términos**: usar la receta de 4 comandos de la guía.
- **kernel equivocado**: seleccionar `env-ia-mlops` en VS Code.
- **no instala**: Codespaces o en pareja; no bloquea la sesión.

EL CASO DEL INSTRUCTOR · ONLINE RETAIL II (UCI, CC BY 4.0)

¿Volverá este cliente en los próximos 90 días?

Los datos

- Facturas de una tienda online, **01/12/2009 – 09/12/2011**
- **1 067 371** filas en dos hojas de Excel
- **5 942** clientes · **53 628** facturas válidas

La decisión que apoya

- ¿A quién enviar una campaña de retención?
- Un ejemplo = **un cliente en una fecha de corte**
- Etiqueta `y = 1` si compra en `[corte, corte+90 d)`

Dos reglas que repetiremos todo el día: **RAW no se toca** y **las variables solo miran hacia atrás**.
Tu caso, Bank Marketing, repite esta receta y se presenta como avance en S2.

ARQUITECTURA DE DATOS

OPERACIONAL → RAW → MASTER → FS → ML

OPERACIONAL	RAW	MASTER	FS	ML
el sistema de ventas	tal cual llegó	depurado y relacional	variables reutilizables	listo para entrenar

OPERACIONAL

`retail.db: clientes · productos · facturas · lineas_factura`

Simula la base con la que funciona la tienda: normalizada, con claves y con la suciedad del origen. La crea el cuaderno 00 desde el archivo de UCI; el pipeline solo la lee.

- 53 628 facturas
- 4 tablas de negocio
- Nadie la limpia

Dos bases SQLite: el origen, `retail.db` (el pipeline solo la lee), y `curso.db`, con prefijos `raw_ master_ fs_ ml_`.

RAW · SIN TOCAR

El dato real viene sucio

19 494	243 007	22 950	6 207	34 335
líneas de facturas canceladas (C…)	líneas sin cliente	cantidad ≤ 0	precio ≤ 0	duplicados exactos

Tipos mezclados

`Invoice` mezcla números y texto («C489449»); `StockCode` también; `Customer ID` viene como decimal.

No son productos

`POST`, `BANK CHARGES`, `M`… ensucian cualquier conteo de ventas.

LINAJE

Una fila, dos capas

raw_lineas_factura

linea_id	1
_carga_id	4
factura_id	489434
codigo_producto	85048
cantidad	12
precio_unitario	6.95
registrada_en	2009-12-01 07:45

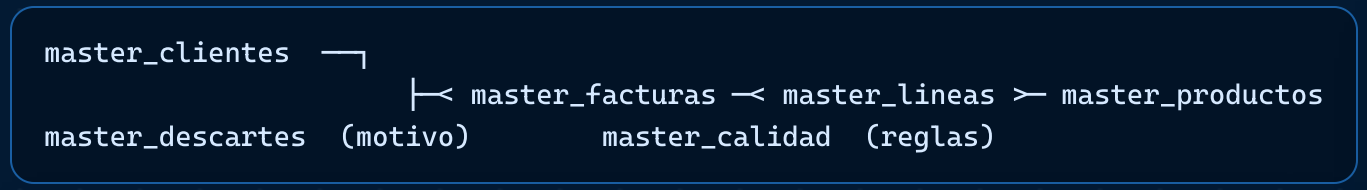
master_lineas + master_facturas

linea_id	1
factura_id	489434
codigo_producto	85048
cantidad	12
precio_unitario	6.95
monto_linea	83.4
es_venta_valida	1
cliente_id	13085 (entero)

El cliente (13085) llega en `raw_facturas` y MASTER lo une a la línea; el monto se calculó (12 × 6.95 = 83.40). El `linea_id` de origen se conserva en las dos capas.

MASTER

Depurar sin perder nada



Reglas de limpieza

Recortar espacios y mayúsculas · descartar duplicados exactos y nulos obligatorios · separar entidades · `es_cancelacion`, `es_producto`, `es_venta_valida`.

Conciliación (dato completo)

RAW	1 067 371
MASTER líneas	1 033 036
Descartes	34 335
MASTER + descartes	1 067 371

1 033 036 + 34 335 = 1 067 371 ✓

CALIDAD

Reglas que se ejecutan, no que se recuerdan

REGLA	VALOR	ESTADO
R01_conciliacion	0	OK
R02_facturas_sin_lineas	0	OK
R03_integridad_referencial	0	OK
R04_monto_coherente	0	OK
R05_fechas_en_rango	0	OK
R06_facturas_con_varios_clientes	0	OK
R07_cancelaciones_con_cantidad_positiva	1	ALERTA

Duras

R01–R05: conciliación, facturas sin líneas, integridad, monto, fechas. Si fallan, el pipeline se detiene.

Alertas e informativas

R06–R07 alertan. R08–R13 miden: % sin cliente (22,76 %), % ventas válidas (97,11 %), duplicados…

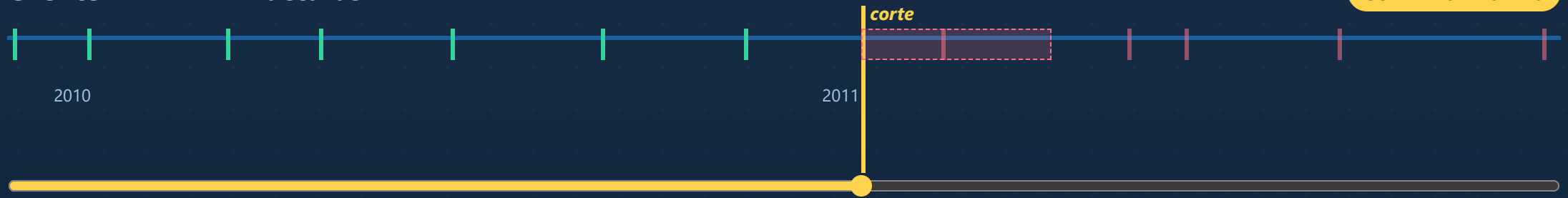
> Una alerta no es un error: es una decisión pendiente de alguien.

FS · VARIABLES POR CLIENTE Y CORTE

Solo se mira hacia atrás

Cliente 12714 · 12 facturas

CORTE 2011-01-10



Facturas usadas por FS (antes del corte)

7

monto acumulado 8520.85 · última 2010-11-16

Ventana de 90 días → objetivo y

1

compra dentro de los 90 días siguientes

FS · FS_CATALOGO

13 variables, 5 familias

Recencia y antigüedad

`recencia_dias` · `antiguedad_dias`

Frecuencia y ritmo

`frecuencia` · `dias_medios_entre_compras` · `frecuencia_90d` · `frecuencia_180d`

Monto

`monto_total` · `ticket_medio` · `monto_90d`

Variedad y contexto

`unidades_total` · `productos_distintos` · `devoluciones` · `es_reino_unido`

La prueba **`test_fs_no_usa_informacion_del_futuro`** borra todo lo posterior al corte y comprueba que las variables no cambian.

ML

Objetivo y partición temporal

TRAIN

cortes 2010-06 a 2010-12 (mensual)

23 760

filas · recompra 48.1 %

VALID

corte 2011-03-01

4 512

filas · recompra 35.1 %

TEST

cortes 2011-06-01 y 2011-09-01

10 132

filas · recompra 38.0 %

Total: **38 404** ejemplos. Antes de construir se comprueba que la ventana de cada corte cabe en los datos (último día: 2011-12-09).

Cada ejemplo = cliente + corte. Un cliente aparece varias veces, una por corte.

MEDIR COMO SE VA A USAR

¿Por qué partir por tiempo?

Partición temporal (test)

AUC 0.796

Entrena con cortes pasados, evalúa en cortes posteriores.

Partición aleatoria (5 semillas)

AUC 0.766

Rango 0.761–0.770. Mezcla clientes de todas las fechas.

Lectura honesta: aquí la aleatoria dio un valor **más bajo**, no más alto. No es una fuga que infle la métrica en este caso.

Se elige la temporal porque **se parece al uso real**: predecir el futuro con lo aprendido del pasado. Además, la tasa de recompra cambia (48 → 35 → 38 %), algo que una mezcla aleatoria esconde.

ENTRENAMIENTO

Tres modelos, una decisión

MODELO	AUC TRAIN	AUC VALID	AUC TEST	PREC. MEDIA TEST
Referencia (sin aprender)	0.500	0.500	0.500	0.380
Regresión logística	0.740	0.786	0.796	0.735
Gradient boosting	0.842	0.761	0.781	0.718

El gradient boosting «sabe» mucho más del pasado (0,842) pero generaliza peor: **sobreajuste**. Gana la regresión logística, más simple.

Con la muestra del curso (23 070 filas) los valores serán distintos: regresión logística valid 0.698 · test 0.747. Eso es esperado, no un error.

Dato completo: 1 067 371 filas. Regla: se elige con valid, se informa test una sola vez.

MLFLOW · TRACKING

Cada ejecución deja huella

EJECUCIÓN	VALID	TEST	HUELLA DATOS
referencia_sin_aprender	0.500	0.500	3521a0421905ba28
regresion_logistica	0.786	0.796	3521a0421905ba28
gradient_boosting	0.761	0.781	3521a0421905ba28
MEJOR-regresion_logistica	0.786	0.796	3521a0421905ba28

Qué se guarda

- parámetros y métricas
- modelo con **firma** y ejemplo de entrada
- huella (sha256) del dato

`mlflow ui` abre la tabla anterior en el navegador. Backend: SQLite.

COOKIECUTTER DATA SCIENCE

El cuaderno es la fuente; el paquete, su traducción

notebooks/

```
00_pipeline_operacional
01_pipeline_raw_a_master
02_pipeline_master_a_fs
03_pipeline_fs_a_ml
04_entrenamiento_y_seguimiento
```

Parametrizados con papermill.

ia_mlops/

```
pipeline_ingesta_raw.py     ← 01
pipeline_ingesta_master.py  ← 01
pipeline_ml.py              ← 02 + 03
pipeline_train.py           ← 04
+ la lógica: dataset · calidad · features
  modeling/ml_dataset · train · predict
```

Cuatro pipelines **independientes e idempotentes**: `python -m ia_mlops.pipeline_ml`… Mismo resultado que el cuaderno.

Prueba de equivalencia: **cuaderno = paquete = script de jupytext** (mismas tablas, mismas métricas). La tarea también se entrega traducida.

PRÁCTICA GUIADA · 135 MIN

Los cinco cuadernos, en orden



00 y 01 · operacional, RAW → MASTER

30 MIN · DESDE EL MIN 10

Crear retail.db con la muestra (00), ingerirla en RAW, inspeccionar descartes, comprobar la conciliación y las reglas de calidad.

Total de la práctica: 135 min.

Guía completa: `01-Sesiones/S1-Datos-y-Entrenamiento/practica/`

ENTREGA

Adapta una variable y decide

Pasos

- Diseña **una variable nueva** en FS (solo con datos anteriores al corte).
- Documéntala en `fs_catalogo` y reconstruye ML.
- Reentrena y **compara dos ejecuciones** en MLflow.
- Escribe una **decisión**: ¿se queda o no? ¿por qué?
- Entrega también la traducción a `.py`.

Aparte: tu **avance en S2** sobre Bank Marketing (RAW → MASTER → FS → ML, entrenamiento y métricas, en cuadernos).

Rúbrica



- **20 %** Reproducibilidad
- **40 %** Técnica (sin fuga, pruebas)
- **40 %** Análisis y decisión

ANTES DE EMPEZAR

Errores que ya vimos

Kernel equivocado

El cuaderno corre con otro Python. Seleccionar `env-ia-mlops`.

Orden de los cuadernos

El 02 necesita el 01. Si falla, ejecutar desde el 01 o usar `make`.

Variable con información del futuro

Si usa datos ≥ corte, la prueba de FS falla. Eso es bueno.

Comparar métricas de datos distintos

Mirar siempre la **huella del dato** en MLflow antes de comparar.

CIERRE

Qué te llevas y qué viene

Hoy

- Cuatro capas con linaje y calidad
- FS sin fuga y partición temporal
- Modelo comparado y registrado en MLflow

S2

- Cargar el modelo desde MLflow (firma)
- API con FastAPI, validación y errores
- Imagen Docker y primer workflow de GitHub Actions

Pendiente para ti: **la tarea**, registrar tus incidencias del entorno y **leer la hoja de tu caso propio** (Bank Marketing).